



Terraform

Terraform

- Відкритий код
- Публічна ліцензія Mozilla 2.0
- Створено Hashicorp (vagrant, consul, packer, vault)
- Розпочато в 2014 році
- Написано в Go
- Підключається

Що таке terraform?

Початкова мета Terraform

Terraform — це інструмент для *створення, зміни та контролю версій* вашої інфраструктури.

Побудова інфраструктури

- Спілкуйтеся з кількома постачальниками хмари/інфраструктури
- Забезпечте створення та послідовність
- Експрес у API-Agnostic DSL



Змінити інфраструктуру

- Застосувати поступові зміни
- Знищувати при потребі
- Попередній перегляд змін
- Легко масштабувати

Контроль версій

- мова HCL
- Файл стану (не зберігайте його в репозиторії git)

Поняття Terraform

Змінні

- Terraform приймає змінні як вхідні дані
- Типові змінні
- JSON, HCL або CLI

Вихід

- Виведіть конкретний текст, IP-адреси, ...
- Console або JSON

Поняття Terraform

Провайдери

- Конфігурація постачальника ресурсів
- наприклад: хмара, провайдер DNS,...
- Часто вимагають URL-адресу, ключ API, ...

Модулі

- Група ресурсів
- Приймає параметри
- Повертає результат

Поняття Terraform

План

- план terraform
- Створює план необхідних змін
- Нічого не робить з інфраструктурою

Застосування

- застосування terraform
- Застосовує план, вносить усі зміни
- За потреби можна скласти план заздалегідь

Поняття Terraform

Файл стану

- Поточний відомий стан інфраструктури
- Зберігається у файлі або зовнішньо
- Блокування

Знищення

- знищити terraform
- Видалити всі ресурси
- ресурси можна "захистити" в конфігурації



Встановити на Linux

Встановити на Linux

Щоб інсталювати Terraform, знайдіть відповідний пакет для вашої системи та завантажте його як zip-архів.

Після завантаження Terraform розархівуйте пакет. Terraform працює як єдиний двійковий файл під назвою terraform. Будь-які інші файли в пакеті можна безпечно видалити, і Terraform працюватиме.

Нарешті переконайтеся, що двійковий файл terraform доступний у вашому PATH. Цей процес буде відрізнятися залежно від вашої операційної системи.

Встановити на Linux

Роздрукуйте розділений двокрапками список місць у вашому PATH.

```
$ echo $PATH
```

Перемістіть двійковий файл Terraform в одне з перелічених місць. Ця команда припускає, що двійковий файл наразі знаходиться у вашій папці завантажень і що ваш PATH містить /usr/local/bin, але ви можете налаштувати його, якщо ваші розташування відрізняються.

```
$ mv ~/Downloads/terraform /usr/local/bin/
```

Перевірте встановлення

Переконайтеся, що встановлення працює, відкривши новий термінальний сеанс і перерахувавши доступні підкоманди Terraform.

```
$ terraform -help
Usage: terraform [-version] [-help] <command> [args]

The available commands for execution are listed below.
The most common, useful commands are shown first, followed by
less common or more advanced commands. If you're just getting
started with Terraform, stick with the common commands. For the
other commands, please read the help and docs before usage.
##...
```

Перевірте встановлення

Додайте будь-яку підкоманду до terraform -help, щоб дізнатися більше про те, що вона робить, і доступні параметри.

```
$ terraform -help plan
```

Усунення несправностей

Якщо ви отримуєте повідомлення про те, що terraform не знайдено, ваша змінна середовища PATH не налаштована належним чином. Будь ласка, поверніться назад і переконайтеся, що ваша змінна PATH містить каталог, де встановлено Terraform.

Включення автодоповнення

Якщо ви використовуєте bash або zsh, ви можете увімкнути завершення табуляції для команд Terraform. Щоб увімкнути автозаповнення, виконайте наступну команду, а потім перезапустіть оболонку.

```
$ terraform -install-autocomplete
```



Hello Terraform

Введение в Terraform

TERRAFORM

```
provider "aws" {  
  // Set AWS_ACCESS_KEY_ID &  
  // AWS_SECRET_ACCESS_KEY env vars  
  region = "us-west-2"  
}  
  
resource "aws_s3_bucket" "my-cool-bucket" {  
  bucket = "my-cool-bucket"  
  acl = "public-read"  
}
```

The name to reference
within terraform files

The name of the bucket

CLOUD FORMATION

```
{  
  "Resources" : {  
    "my-cool-bucket" : {  
      "Type" : "AWS::S3::Bucket",  
      "Properties" : {  
        "AccessControl" : "PublicRead"  
      }  
    }  
  }  
}
```

Generated name:
stackname-my-cool-bucket-abc123id

ПЛАН TERRAFORM

MAIN.TF

```
# Configure aws with a default region
provider "aws" {
  region = "us-east-1"
}

# Create a demo s3 bucket
resource "aws_s3_bucket" "chadev_tf_demo" {
  bucket = "chadev-tf-demo"

  tags {
    Name = "Terraform demo for ChaDev"
    Environment = "Prod"
    TF = "yes"
  }
}
```

TERRAFORM PLAN

```
+ aws_s3_bucket.chadev_tf_demo
  acl: "" => "private"
  arn: "" => "<computed>"
  bucket: "" => "chadev-tf-demo"
  force_destroy: "" => "0"
  hosted_zone_id: "" => "<computed>"
  region: "" => "<computed>"
  tags.#: "" => "3"
  tags.Environment: "" => "Prod"
  tags.Name: "" => "Terraform demo for ChaDev"
  tags.TF: "" => "yes"
  website_domain: "" => "<computed>"
  website_endpoint: "" => "<computed>"
```

Plan: 1 to add, 0 to change, 0 to destroy.

ЗАСТОСУВАТИ ТА ВСТАНОВИТИ TERRAFORM

TERRAFORM APPLY

```
aws_s3_bucket.chadev_tf_demo: Creating...
  acl:      "" => "private"
  arn:      "" => "<computed>"
  bucket:   "" => "chadev-tf-demo"
  force_destroy: "" => "0"
  hosted_zone_id: "" => "<computed>"
  region:   "" => "<computed>"
  tags.#:   "" => "3"
  tags.Environment: "" => "Prod"
  tags.Name: "" => "Terraform demo for ChaDev"
  tags.TF:   "" => "yes"
  website_domain: "" => "<computed>"
  website_endpoint: "" => "<computed>"
aws_s3_bucket.chadev_tf_demo: Creation complete

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

State path: terraform.tfstate
```

TERRAFORM STATE

```
{
  "version": 1,
  "serial": 1,
  "modules": [
    {
      "path": [
        "root"
      ],
      "outputs": {},
      "resources": {
        "aws_s3_bucket.chadev_tf_demo": {
          "type": "aws_s3_bucket",
          "primary": {
            "id": "chadev-tf-demo",
            "attributes": {
              "acl": "private",
              "arn": "arn:aws:s3:::chadev-tf-demo",
              "bucket": "chadev-tf-demo",
              "cors_rule.#": "0",
              "force_destroy": "false",
              "hosted_zone_id": "Z3AQBSTGFYJSTF",
              "id": "chadev-tf-demo",
              "policy": "",
              "region": "us-east-1",
              "tags.#": "3",
              "tags.Environment": "Prod",
              "tags.Name": "Terraform demo for ChaDev",
              "tags.TF": "yes",
              "website.#": "0"
            }
          }
        }
      }
    }
  ]
}
```

TERRAFORM STATE

- JSON
- Оновлюється під час запуску Terraform
- Оновлюється перед операцією
- Зворотна сумісність між версіями TF

ВИКОРИСТАННЯ ВІДДАЛЕНОГО СТАНУ ЯК ЗАЛЕЖНОСТІ

TANGENT: USING REMOTE STATE AS A DEPENDENCY

```
resource "terraform_remote_state" "core" {  
  backend = "s3"  
  config {  
    bucket = "a-tf-state-bucket"  
    key    = "hosted/core.json"  
    region = "us-east-1"  
  }  
}  
  
resource "aws_route53_record" "my-awesome-subdomain.yhat.com" {  
  zone_id = "${terraform_remote_state.core.output.yhat-zone-id}"  
  name    = "my-awesome-subdomain.yhat.com"  
  type    = "A"  
  ttl     = "60"  
  records = [  
    "${aws_eip.my-elastic-ip.public_ip}"  
  ]  
}
```



ДАВАЙТЕ
ТЕРАФОРМУВАТИ
СЕРВЕР

СТВОРЕННЯ СЕРВЕРА

```
# Create a key pair for our servers
resource "aws_key_pair" "chadev_demo" {
    key_name = "chadev_demo_key"
    public_key = "ssh-rsa ..."
}

# Create a demo server
resource "aws_instance" "chadev_demo_server" {
    # CentOS 7 (x86_64) with Updates HVM
    ami = "ami-6d1c2007"
    instance_type = "t2.medium"
    key_name = "${aws_key_pair.chadev_demo.key_name}"
    tags {
        Name = "ChaDev Demo Server"
        TF = "yes"
    }
}
```



Get the name of the key by referencing the output of the `aws_key_pair`. Notice we're using the variable name we define.

ПЛАН TERRAFORM

```
+ aws_instance.chadev_demo_server  
...
```

```
+ aws_key_pair.chadev_demo  
...
```

```
Plan: 2 to add, 0 to change, 0 to destroy.
```

СТВОРЕННЯ СЕРВЕРА

```
Error applying plan:
```

```
1 error(s) occurred:
```

```
* aws_instance.chadev_demo_server: Error launching source instance: VPCResourceNotSpecified: The specified instance type can only be used in a VPC. A subnet ID or network interface ID is required to carry out the request.  
   status code: 400, request id:
```

Terraform не виконує автоматичний відкат у разі виникнення помилок. Натомість ваш файл стану Terraform було частково оновлено всіма ресурсами, які успішно завершено. Усуньте помилку вище та подайте заявку ще раз, щоб поступово змінити свою інфраструктуру.

```
Plan: 1 to add, 0 to change, 0 to destroy.
```



ДАВАЙТЕ
ТЕРАФОРМУВАТИ
VPS

ЧАСТИНИ VPC

- Визначення VPC (блок CIDR)
- Підмережі
- Інтернет-шлюз, який дозволяє нашій підмережі маршрутизувати до Інтернету
- Таблиця маршрутизації, щоб визначити, як маршрутизується трафік
- Асоціація основної таблиці маршрутів, щоб повідомити vpc, яку таблицю маршрутів використовувати за замовчуванням

VPC

```
#####  
# VPC  
#####  
  
resource "aws_vpc" "chadev_demo" {  
  cidr_block = "10.20.0.0/16"  
  enable_dns_hostnames = true  
  
  tags {  
    Name = "ChaDev Demo VPC"  
    Env = "Prod"  
    TF = "yes"  
  }  
}
```

ПІДМЕРЕЖА

```
# Create a subnet for our instances
resource "aws_subnet" "chadev_demo_1a" {
  vpc_id = "${aws_vpc.chadev_demo.id}"
  cidr_block = "10.20.0.0/24"
  availability_zone = "us-east-1a"
  map_public_ip_on_launch = true

  tags {
    Name = "ChaDev Subnet AZ 1a"
    Env = "Prod"
    TF = "yes"
  }
}
```

ІНТЕРНЕТ ШЛЮЗ

```
# Create an internet gateway so we can talk to the world
resource "aws_internet_gateway" "main" {
  vpc_id = "${aws_vpc.chadev_demo.id}"

  tags {
    Name = "ChaDev VPC IGW"
    Env = "Prod"
    TF = "yes"
  }
}
```

МАРШРУТНА ТАБЛИЦЯ

```
# Replace the default routing table with one explicitly defined here
resource "aws_route_table" "chadev_demo" {
  vpc_id = "${aws_vpc.chadev_demo.id}"

  # Send all traffic not matched by other routing rules to the internet
  route {
    cidr_block = "0.0.0.0/0"
    gateway_id = "${aws_internet_gateway.main.id}"
  }

  tags {
    Name = "ChaDev Main RTB"
    Env = "Prod"
    TF = "yes"
  }
}
```

АСОЦІАЦІЯ МАРШРУТНОЇ ТАБЛИЦІ

```
# Assign our route table as the default route table
resource "aws_main_route_table_association" "main" {
  vpc_id = "${aws_vpc.yhat-tf-demo.id}"
  route_table_id = "${aws_route_table.yhat-tf-default.id}"
}
```

ОРГАНІЗАЦІЯ ФАЙЛІВ TERRAFORM

- Terraform шукає всі файли із розширенням .tf
- Я особисто додаю до файлів префікс постачальника, де це можливо
- Давайте розмістимо vpc в aws.vpc.tf
- Давайте розмістимо решту в aws.demo.tf

ЗАСТОСУВАННЯ ТЕРАФОРМ

```
Error applying plan:
```

```
1 error(s) occurred:
```

```
* aws_instance.chadev_demo_server: Error launching source instance: VPCResourceNotSpecified: The specified instance type can only be used in a VPC. A subnet ID or network interface ID is required to carry out the request.
```

```
    status code: 400, request id:
```

ВКАЖІТЬ ПІДМЕРЕЖУ ТА ГРУПИ БЕЗПЕКИ

```
vpc_security_group_ids = [  
    "${aws_security_group.chadev_demo.id}",  
]  
  
subnet_id = "${aws_subnet.chadev_demo_1a.id}"
```

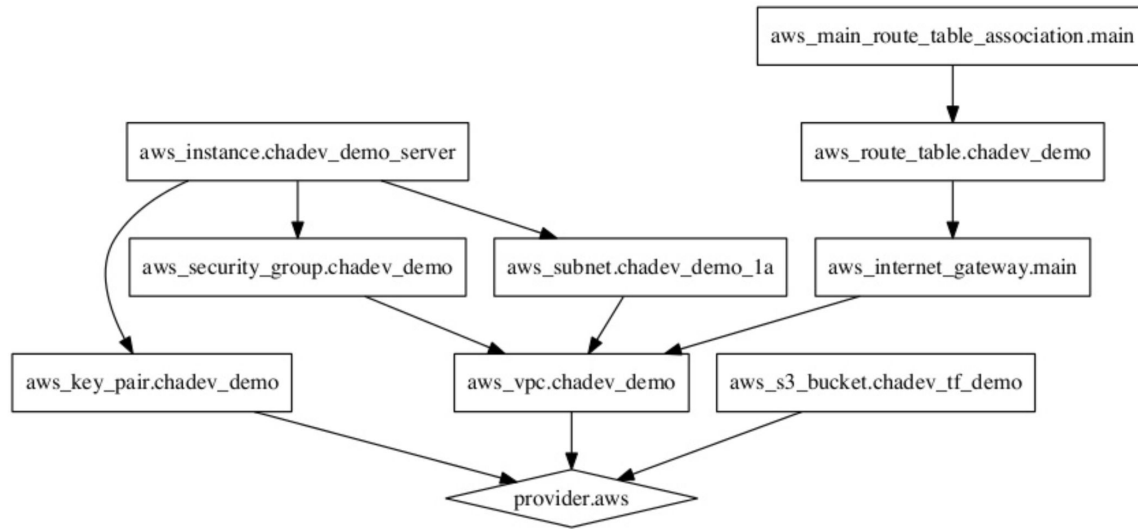


ЗАПУСК І ПЕРЕГЛЯД

TERRAFORM БУДЕ ГРАФІКОМ ЗАЛЕЖНОСТЕЙ

```
▶ terraform graph | dot -Tpng > graph.png
```

ΓΡΑΦΙΚ TERRAFORM | DOT -TPDF > GRAPH.PDF



Питання та відповіді